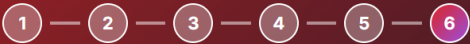


Learning Week: Day 1 → Day 6

A complete guide to understanding chains, agents, memory, orchestration, and building a real retail data pipeline



Day 1: Chains

Day 2: Templates

Day 3: Memory & Tools

Day 4: LangGraph

Day 5: Orchestrator

Day 6: The Project

Day 2: Prompt Templates & Output Parsers

Reusable Prompts + Structured Outputs

Prompt Templates

A prompt template lets you write a prompt once with placeholders like `{product_name}` and reuse it with different values. Instead of hardcoding every detail, you create a template and inject variables at runtime.

TEMPLATE EXAMPLE:

```
template = """You are a marketing expert.
Write a 2-sentence product description for: {product_name}
Target audience: {audience}"""

# Use multiple times
result1 = template.format(product_name="Laptop", audience="Techies")
result2 = template.format(product_name="Phone", audience="General")
```

Output Parsers

By default, an LLM replies in free-flowing text — hard for code to use reliably. An output parser instructs the LLM to reply in a specific structure (like JSON) so your code can read fields directly.

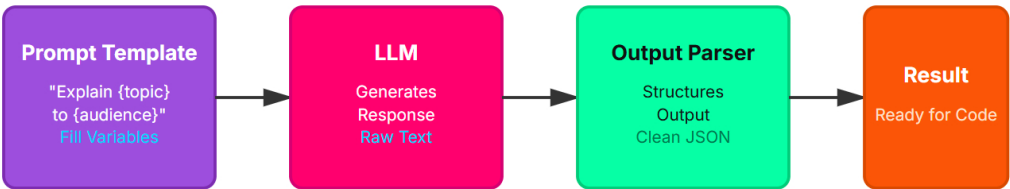
WITHOUT PARSER (MESSY):

```
response = "The product costs $99 and has 4.5 stars"
# How do you extract price? Regex? Fragile!
```

WITH JSONOUTPUTPARSER (CLEAN):

```
response = {"price": 99, "rating": 4.5}
price = response["price"] # Easy!
```

Template → LLM → Parser Flow



Key Concepts

PromptTemplate: Create reusable prompts with variables

Why This Matters

Parsers make your application **reliable**. Instead of hoping the LLM's response contains what you need, you enforce a contract: "reply with exactly these fields in this format."

StrOutputParser: Returns response as plain text string

JsonOutputParser: Returns response as valid JSON object

Pydantic Models: Define expected output structure precisely

Structured Output: Data your code can reliably parse

Example: `customer_service_bot.py` and `customer_service_bot.py`

Example: A customer service bot that extracts sentiment, urgency, and category from emails — without a parser, you'd spend time writing regex. With a parser, the LLM does it right.

Key Takeaway: Prompt templates make your code DRY (Don't Repeat Yourself). Output parsers make your code reliable. Together, they transform unstructured LLM responses into data your application can trust and use.

Free Resources to Dive Deeper

LangChain Docs

Official documentation with API reference and guides

LangChain Academy

Free official courses on LangChain fundamentals

LangGraph Docs

Guide to building graphs and agents

Multi-Agent Tutorial

Real example of orchestrator agents in action

🌟 Learning Week: LangChain + LangGraph Fundamentals

DevSynt AI Automation Internship · Mentor: Afnan Shoukat

Quiz coming next — make sure you understand these concepts!